
3. Fast Lane

Deterministic only. No model call, no network, no variance.

PII (`controlplane/fastlane/pii.py`)

Pattern matching with checksum validation: Aadhaar (Verhoeff), card numbers (Luhn), PAN, email, Indian mobile numbers, and a deliberately weak capitalised-bigram name detector that only fires at `strict`.

The rule that matters is not "does the response contain a phone number" but "**does it contain personal data the user did not already have**". An identifier the user supplied in this conversation is suppressed, compared on digits so that `98214 32210` and `+91 98214 32210` are recognised as the same thing. Echoing a customer's own details back is permitted by the account handling policy; disclosing somebody else's is the leak.

Known failure mode: long numeric identifiers — order numbers, invoice references — match the mobile-number pattern. This is the main source of the detector's false positives, and two such cases are in the corpus on purpose.

Deny lists (`controlplane/fastlane/lists.py`)

Phrases that commit the enterprise to something an assistant cannot commit to ("I guarantee", "I'll waive", "no inspection needed"), each with a negation guard so that "I'm not able to waive" does not fire.

Loop and budget breakers (`controlplane/fastlane/loops.py`)

Identical tool calls above a threshold, total tool calls above budget, and revealed-preference signals in the user's own words ("no, not that", "I said"). Multi-turn agents compound risk; these are cheap circuit breakers on that compounding.

Known limitation: these are deterministic rules over deterministic signals. Their perfect scores on the corpus reflect the rule, not a learned model, and should be read that way.

4. Slow Lane

Grounding / hallucination (`controlplane/slowlane/grounding.py`)

We never claim a statement is false. There is no real-time ground truth for an arbitrary claim — the brief says so, and it is true. What an enterprise does have is a set of documents it has agreed to be bound by. So the check reports that a claim is **unsupported by**, or **contradicts**, those documents.

Per claim:

1. Skip claims the corpus cannot adjudicate. An order-status fact is not a policy assertion, and flagging it is noise.
2. Find the closest sentence in the governed corpus, plus the retrieved context.
3. A figure is **grounded** if the source document the claim most resembles asserts it, or if it came from the case data in front of the model. Grounding is scoped to that document: a shipping SLA of 3–5 days must not be allowed to vouch for a refund window of 5 days just because both numbers exist somewhere in the corpus.
4. An ungrounded figure in a claim that closely restates a source is a **contradiction** (high severity). One in a claim that matches nothing in particular is **unsupported**.

5. Separately, if the contract sets `grounding_required`, an answer with no supporting document at all is a finding on its own terms — however confident it sounds.

Known failure mode, measured: the offline judge is built around figures. Open-ended fabrication with nothing numeric to contradict — an invented loyalty tier, an invented benefit — is caught only by the `grounding_required` control, and not always. Recall on the hand-written holdout is **0.75** against **0.887** on the generated corpus, and the gap is almost entirely this class. This is the clearest case in the system for routing to an LLM judge; the adapter is built (§6) and this is what it is for.

Counterfactual bias screen (`controlplane/slowlane/bias.py`)

Hold every non-demographic input constant, change one protected attribute, compare the outputs. Three independent channels:

CHANNEL	FIRES WHEN	CONFIDENCE
Decision flip	the recommendation itself changes	0.95
Confidence movement	the stated confidence moves more than the noise floor (0.03)	scaled to 0.92
Proxy language	the counterfactual justification uses stand-in language ("profile", "less typical") the baseline did not	0.72
Cited factors	the counterfactual cites a different set of permitted factors	0.48

This is a screening signal, not a finding of discrimination. Divergence means the case needs a human. It is not proof that the system is biased, and every surface that displays it says so. A single trace cannot establish disparate treatment; only a pattern across many can, and even then only a human can make that call.

In production the counterfactual comes from a shadow call to the same model. In this prototype it is pre-recorded on the trace so the screen is exactly reproducible offline.

Known limitation: the screen tests the attributes we thought to swap. It cannot find a proxy we did not anticipate.

Explanation policy (`controlplane/slowlane/policy.py`)

For decision support: an adverse recommendation justified by a protected attribute, by a proxy for one, or by nothing at all. Adverse-action rules require the specific permitted factor, its observed value and the threshold — so "profile", "the area they live in" and "background" are violations in exactly the way that naming the attribute outright is.

5. Multi-label fusion (`controlplane/risk.py`)

Bias, hallucination and privacy overlap in practice. The brief's own example is a fabricated detail about a person, which is simultaneously a hallucination and a privacy concern.

We resolve this with evidence rather than with wording. Personal data that appears **in the retrieved context** is real data being disclosed: a privacy event. Personal data that appears **nowhere in context** was invented by the model: a privacy event *and* a hallucination — and strictly worse, because there is no source to correct.

Both labels are recorded. **One action** is taken, and it is the strictest one any finding demands, because the user receives one response and the enterprise takes one action. A jurisdiction can additionally force human review for a whole risk family regardless of that instance's severity.

6. The judge backend (`controlplane/slowlane/judge.py`)

Two rules:

1. **The evaluator is never the generator marking its own homework.** The Anthropic backend is a different model family from the assistant being evaluated. The offline backend is not a language model at all.
2. **The default must run with no API key and no network**, so that every number in `outputs/` is reproducible by anyone who clones the repo.

`OfflineJudge` is retrieval similarity plus numeric agreement. It is described as that everywhere it appears, and never as an LLM judge. Set `MYCROFT_JUDGE=anthropic` with a key to route semantic evaluation to Claude instead; the checkability filter and the fallback path are unchanged, so only the verdict source moves. If the API call fails the judge falls back to the offline path — it fails safe, never open.

7. What the labels mean

Ground truth in `data/traces.jsonl` is **correct by construction**: each scenario template knows which risk it plants. That is a sound basis for precision and recall *against this corpus*, and its limitation is that a detector could in principle fit the templates rather than the risk.

`data/holdout.jsonl` exists to test that. Its 30 traces were written by hand **after** the detectors were built, deliberately awkward: negated policy language, identifiers in unusual formats, correct figures written as words, fabrications with no numbers to contradict, and counterfactual divergences placed just either side of the threshold. Where holdout scores are worse, that gap is the honest estimate of how much the generated numbers flatter us.

Neither set is production traffic. Nothing here estimates production performance.

8. What a fix can and cannot do

A contract change reduces what **escapes**, not what the model gets wrong. We measure and report escape reduction and never call it an accuracy improvement.

Every fix costs something. `controlplane/lifecycle.py` measures the additional false positives a change introduces across the whole use case, and the change in human review load, and reports both next to the benefit. In the demo loop that trade is: **escapes halved, at +11 false positives and +4 human reviews across 114 interactions.**

Some defects have no contract fix at all. A tool-call loop is an orchestration bug; the control plane's job there is to hold the regression test that proves it was fixed, and `prescribe()` says so rather than inventing a configuration change that would not help.